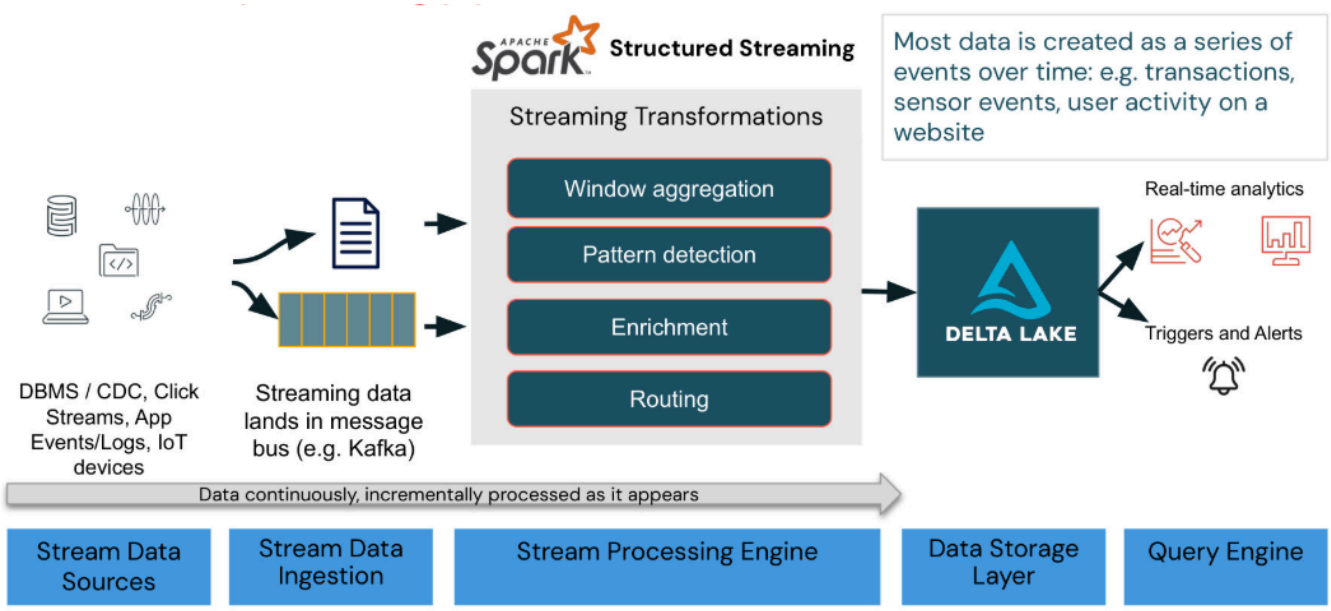
# swap47- Databricks Streaming and Lakeflow Declarative Pipelines course study

Last updated by | Swapnil4 Mane | Sep 3, 2025 at 11:22 AM GMT+5:30

Contents


- Stream Processing Architecture
- Batch vs Streaming
- Streaming Data Concepts
  - A) stateless vs stateful
    - a) processing vs event time out
    - b) Memory impact of state management in stateful stream...
      - i) Unbounded State
      - ii) Watermarking
      - iii) Event Time constraints
  - B) Rate Limiting
  - C) Output mode in writeStream
  - Append Mode
  - Complete Mode
- Streaming joins
- Streaming ETL patterns with lakeflow declarative pipelines
  - a) Data Ingestion Patterns
    - i) Simplex
    - ii) Multiplex
  - b) Data Quality Enforcement
    - i) Constraints (non DLT/ normal delta)
    - ii) Expectations (DLT)


## Stream Processing Architecture




This architecture shows how streaming data from various sources (DBMS, CDC, IoT, clickstreams, etc.) is ingested through a message bus like Kafka.
The data is then processed in Apache Spark Structured Streaming**, applying transformations such as window aggregation, pattern detection, enrichment, and routing.
Processed data is stored in Delta Lake, enabling reliable storage and integration with downstream systems.
Finally, it powers real-time analytics, queries, alerts, and triggers for business use cases.

## Batch vs Streaming

read data from path (hdfs or volume) one sample code
stream waits for next batch, as in next record which arrives on that path.
Assume one parquet file is placed either on hdfs path (like in our On Premise) or in Volume (in Databricks)
Batch sample code-

```
# ===== Read Batch Data =====
input_path = "/path/to/input/data"
df = spark.read.format("parquet").load(input_path)

# ===== Display Data =====
df.show(10, truncate=False)

# ===== Write Batch Data to a Table =====
# Creates or overwrites the table
table_name = "my_database.my_table"
df.write.format("parquet").mode("overwrite").saveAsTable(table_name)
```

Streaming sample code-

```
# ===== Streaming Read =====
input_path = "/Volumes/catalog/schema/table_name"   # Databricks Volumes path (or can also use same hdfs pa
df = spark.readStream.format("parquet").load(input_path)

# ===== Display Streaming Data =====
# In Databricks, you can use display(df) directly for live updates
display(df)

# ===== Streaming Write to Table =====
table_name = "my_database.my_table"

query = df.writeStream \
    .format("delta") \                  # Delta is recommended for tables
    .option("checkpointLocation", "/Volumes/catalog/schema/checkpoints/my_table_cp") \
    .outputMode("append") \ # Creates/updates table continuously
    .trigger(processingTime = "0 seconds") \
    .toTable(table_name)

query.awaitTermination()
```

Here, same parquet file can be placed on a path, but batch DF executes only once whilst streaming DF waits for next batch, as in next record to arrive on input path.
In Databricks, tables are stored in Delta format

In Structured Streaming, triggers define when the system should process available data. Since streaming data is continuous, Spark treats it as a series of small batches. A trigger controls how often Spark stops to check for new data, processes all data that has arrived up to that point, and then moves on to the next batch.

## Streaming Data Concepts

### A) stateless vs stateful

**Stateless**-
A stateless streaming job means you're just consuming the stream and applying transformations that don't depend on historical state (e.g., simple filters, column projections, format conversion).
No aggregations or state tracking

```
# ===== Streaming Read (from Volumes in Databricks) =====
input_path = "/Volumes/catalog/schema/input_data"
df = spark.readStream.format("json").load(input_path)

# ===== Stateless Transformations =====
# Just filter and select columns (no aggregation, no state)
filtered_df = df.filter(col("status") == "active").select("id", "status", "event_time")

# ===== Streaming Write =====
query = filtered_df.writeStream \
    .format("delta") \
    .option("checkpointLocation", "/Volumes/catalog/schema/checkpoints/stateless_cp") \
    .outputMode("append") \
    .toTable("my_database.stateless_events")

query.awaitTermination()
```

In stateful streaming, Spark maintains memory of **past events** across batches, using operations like **window aggregations, deduplication, joins, or mapGroupsWithState**.
So stateful streaming operations are-

```
# ===== Streaming Read (from Volumes in Databricks) =====
input_path = "/Volumes/catalog/schema/input_data"
df = spark.readStream.format("json").load(input_path)

# ===== Stateful Transformation: Windowed Aggregation =====
# Count events per user in 5-minute windows
agg_df = df.groupBy(
    window(col("event_time"), "5 minutes"),
    col("user_id")
).agg(count("*").alias("event_count"))

# ===== Streaming Write =====
query = agg_df.writeStream \
    .format("delta") \
    .option("checkpointLocation", "/Volumes/catalog/schema/checkpoints/stateful_cp") \
    .outputMode("append") \
    .toTable("my_database.user_event_counts")

query.awaitTermination()
```

### a) processing vs event time out

Sample data example-

| Record | event_time | processing_time (arrival) |
|---|---|---|
| R1 | 12:00:05 | 12:00:10 |
| R2 | 12:00:20 | 12:00:22 |
| R3 | 12:00:40 | 12:03:00 (late arrival) |
| R4 | 12:01:05 | 12:01:07 |

**Processing timeout**- when Spark received/processed the record

```
If trigger = 1 min
Batch1 → [12:00:00 – 12:01:00] gets R1, R2

Batch2 → [12:01:00 – 12:02:00] gets R4

R3 arrives at 12:03:00 → it will be processed in the 12:03 batch (even though its event_time was 12:00:40).
```

Late events are not special — they're just processed when they arrive.

**Event timeout**- when the event actually happened (inside the data)
Example:

```
 With a 2-minute watermark on event_time:
Window [12:00:00 – 12:01:00] waits until watermark passes 12:03:00 before closing.

R1 and R2 → fall in the 12:00–12:01 window.

R3 (event_time=12:00:40) arrives late at 12:03 → still included in that same window (because watermark allo

R4 (12:01:05) → goes into the 12:01–12:02 window.
```

Late events can still be counted in the correct window, until watermark closes it.**

### b) Memory impact of state management in stateful streaming

Aggregations using past known records are used mostly for stateful streaming and for that state needs to be stored of data which had appeared in previous batches.
Note: State is saved in memory, and not written in a table, or path

Illustration of state memory usage:
State query- `df.groupBy("user_id").count()`
Data arrives continuously with event_time.
State = Spark's in-memory hash tables storing counts.
Checkpoint = backing store of this state.

#### i) Unbounded State

if no constraint given, state remains unbounded

```
Every unique `user_id` ever seen stays in state.

Example:
12:00 → 100 MB state (1M users)

12:10 → 250 MB state (2.5M users)

12:30 → 800 MB state (8M users)

   Memory keeps growing indefinitely.
   Risk: Out-of-memory or huge checkpoint files.
```

Can check/monitor ever growing state memory size-
On Premise- in checkpoint/state hdfs path
In Databricks, can see raw data value.

As the stream runs, the size of streaming state will keep growing indefinitely as all past input must be saved since, any new input can match with any input from the past.

#### ii) Watermarking

Apply watermark, removes from state any data which comes late.

```
df.withWatermark("event_time", "5 minutes")
Spark keeps only data within max_event_time - 5 min.

Example:

12:00 → 100 MB (1M users in last 5 min)

12:10 → 120 MB (still ~1.2M users)

12:30 → 150 MB (old state dropped, but still growing slowly if many active users)

Memory grows slower, but if active key cardinality is high, still heavy.
```

Events removed from state are based on
eventTime < Max event time seen - late threshold
This saves memory a bit but still not that efficient as it's only useful for removing of late incoming records.

**iii) Event Time constraints**

```
df.filter(col("event_time") >= current_time - expr("INTERVAL 5 MINUTES"))
Only process recent 5-minute slice.

Once interval passes, Spark can drop all state for old windows.

Example:

12:00–12:05 → 100 MB

12:05–12:10 → ~100 MB (old dropped, new interval fresh)

12:30–12:35 → ~100 MB
 State stays bounded and predictable hence state memory size remains around the same (reset each interval).
```

Also, state can be removed in flatMapGroupsWithState operations using-

```
if (state.hasTimedOut && state.exists) {

   val s = state.get

   state.remove()
}
```

## B) Rate Limiting

**`maxFilesPerTrigger`**
Available in **all Spark Structured Streaming** file sources.
Controls **how many new files** will be processed in each micro-batch.
Example: If 100 files land in the directory and `maxFilesPerTrigger=10`, Spark processes 10 files per batch → 10 micro-batches in total.

**`maxBytesPerTrigger`**
Not available in the generic Spark file streaming source.
This option is supported only in Databricks Auto Loader (cloudFiles) and Delta streaming sources.
It lets you cap the total data volume (in bytes) processed per micro-batch, e.g., `maxBytesPerTrigger=134217728` (128 MB).
This is more flexible than `maxFilesPerTrigger`, since file sizes can vary.

## C) Output mode in writeStream

Eg. Input Stream-

| user_id | region | event_time |
|---|---|---|
| U1 | US | 12:00:01 |
| U2 | EU | 12:00:05 |
| U3 | US | 12:00:08 |
| U1 | US | 12:01:02 |

append vs complete
append for raw ingestion (bronze layer primarily)
complete for gold layer-

- The **entire result table is rewritten** on every trigger.
- Works only when your query result is **aggregated** and you want the **latest snapshot**.

### Append Mode

```
df.writeStream.outputMode("append").toTable("bronze_events")
```

Each trigger writes only the new incoming rows.

After trigger 1:

| user_id | region | event_time |
|---|---|---|
| U1 | US | 12:00:01 |
| U2 | EU | 12:00:05 |

After trigger 2:

| user_id | region | event_time |
|---|---|---|
| U1 | US | 12:00:01 |
| U2 | EU | 12:00:05 |
| U3 | US | 12:00:08 |
| U1 | US | 12:01:02 |

Raw data preserved → great for **Bronze layer** in Databricks.

### Complete Mode

```
agg = df.groupBy("region").count()
agg.writeStream.outputMode("complete").toTable("gold_region_counts")
```

Each trigger rewrites the entire aggregated result table.

After Trigger 1 (events U1, U2)

Output table:

| region | count |
|---|---|
| US | 1 |
| EU | 1 |

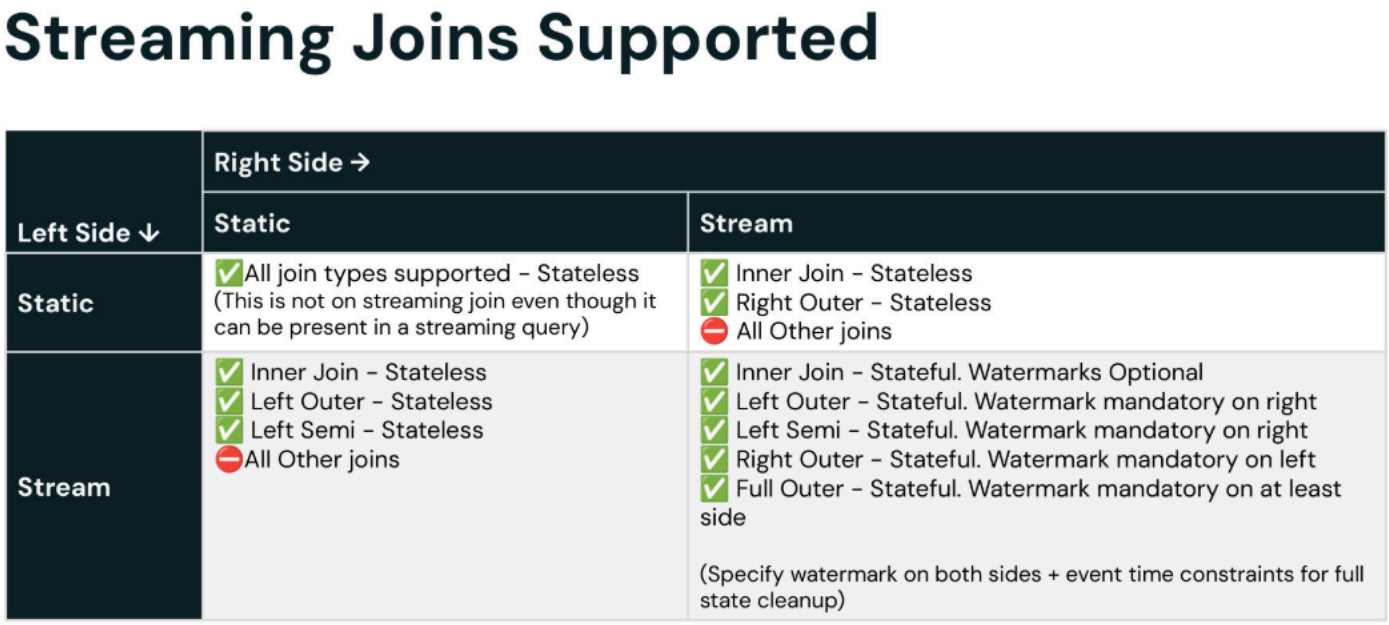
After Trigger 2 (new events U3, U1)

Output table is recomputed fully:

| region | count |
|---|---|
| US | 3 |
| EU | 1 |

Always has the latest snapshot of metrics → ideal for **Gold layer** in Databricks.

## Streaming joins




Note: Anti joins don't work in streaming dataframes or streaming/static dataframes.
To use left anti / right anti joins, use foreachBatch and define a method inside which anti joins can be used.
Eg. Load deduplicated records in master table from a streaming source

```
# Streaming DataFrame (continuous input source)
streamDF = (spark.readStream
                 .format("kafka")
                 .option("kafka.bootstrap.servers", "host:port")
                 .option("subscribe", "topic")
                 .load())

# Process each batch
def process_batch(batchDF, batchId):
    masterDF = spark.read.format("parquet").load("master_path") # master table's path having data till prev
    (batchDF.join(masterDF, ["id"], "left_anti")
           .write.mode("append")
           .saveAsTable("master_table"))

# Write stream with foreachBatch
query = (
    streamDF.writeStream
    .foreachBatch(process_batch)
    .outputMode("append")
    .start()
)
```

# Streaming ETL patterns with lakeflow declarative pipelines

## a) Data Ingestion Patterns

Simplex vs multiplex

### i) Simplex

Number of sources don't matter, but each source is streamed in its own delta table

```
# One topic → one job
df = (spark.readStream
      .format("kafka")
      .option("subscribe", "transactions")
      .load())

df.writeStream.format("delta").option("checkpointLocation", "...").table("bronze_transactions")
```

### ii) Multiplex

- One pipeline handles multiple topics.
- Centralized monitoring via `dlt.expect*`.
- Easy SLA [Service Level Agreement] split (you can add `@dlt.expect_or_fail` if you want the pipeline to **stop** on critical issues for one branch).
  Ingestion happens at bronze layer, several types of data independently streamed in same delta table

Requires routing logic

```
# Multiple topics → one job
df = (spark.readStream
      .format("kafka")
      .option("subscribePattern", "topic.*")
      .load())

# Route by topic
from pyspark.sql.functions import col

transactions = df.filter(col("topic") == "transactions")
customers = df.filter(col("topic") == "customers")

transactions.writeStream.format("delta").option("checkpointLocation", ".../txn").table("bronze_transactions
customers.writeStream.format("delta").option("checkpointLocation", ".../cust").table("bronze_customers")
```

Using DLT-

```
import dlt
from pyspark.sql.functions import col, from_json

@dlt.table
def bronze_raw():
    return (
        spark.readStream.format("kafka")
            .option("subscribePattern", "topic.*")
            .load()
    )

@dlt.table
@dlt.expect_or_fail("valid_txn", "value IS NOT NULL")
def bronze_transactions():
    return (
        dlt.read_stream("bronze_raw")
            .filter(col("topic") == "transactions")
            .selectExpr("CAST(value AS STRING) as json")
    )

@dlt.table
def bronze_customers():
    return (
        dlt.read_stream("bronze_raw")
            .filter(col("topic") == "customers")
            .selectExpr("CAST(value AS STRING) as json")
    )
```

## b) Data Quality Enforcement

### i) **Constraints** (non DLT/ normal delta)

Validating data in Spark code by filtering records before writing.

```
# Apply constraint: filter only valid ages (0–120)
valid_df = df.filter((col("age") >= 0) & (col("age") <= 120))

# Invalid rows can be saved separately if needed
invalid_df = df.filter(~((col("age") >= 0) & (col("age") <= 120)))

# Write valid records to Delta
valid_df.writeStream.format("delta").outputMode("append").option("checkpointLocation", "/chk/valid").start(

# Write invalid records to quarantine table
invalid_df.writeStream.format("delta").outputMode("append").option("checkpointLocation", "/chk/invalid").st
```

Quarantine table can be helpful for analytics

### ii) **Expectations** (DLT)

DLT provides three levels of handling when an expectation is applied:

`@dlt.expect` → just log violations, keep records.

```
@dlt.expect("valid_age", "age >= 18")
@dlt.table
def bronze_users():
    return (
        spark.readStream.format("json").load("/data/users")
    )
If age < 18, record still lands in the table, but a metric is logged (you can monitor in the UI)
```

`@dlt.expect_or_drop` → drop bad records, continue pipeline.

```
@dlt.expect_or_drop("non_null_email", "email IS NOT NULL")
@dlt.table
def silver_users():
    return dlt.read("bronze_users")

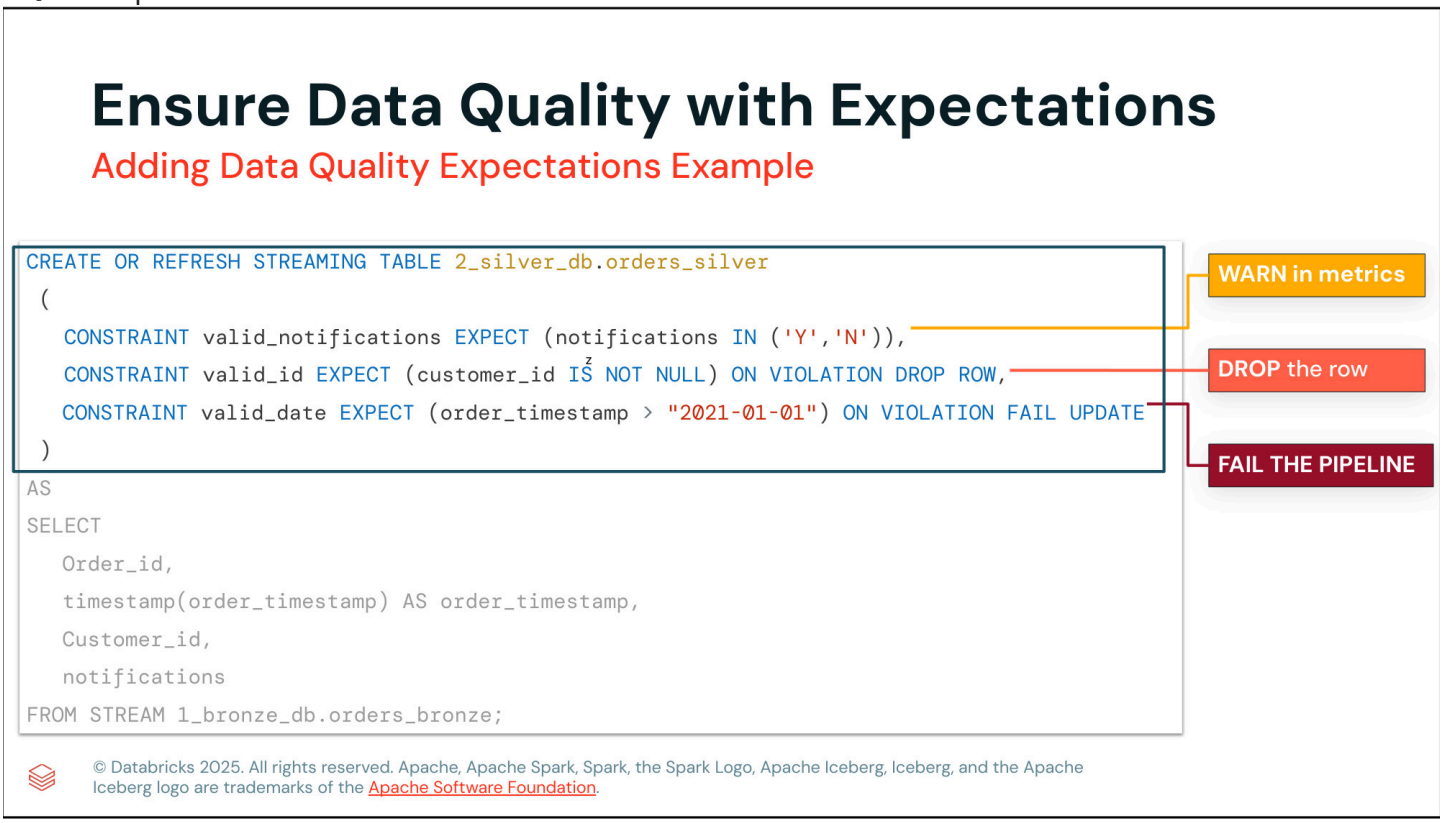
Any row with `NULL` in email will be excluded from silver_users
```

`@dlt.expect_or_fail` → fail pipeline if violations occur.

```
@dlt.expect_or_fail("must_have_id", "id IS NOT NULL")
@dlt.table
def gold_users():
    return dlt.read("silver_users")

Fail pipeline if even one record has id = NULL
```

SQL example-




Advantages of Expectations over Manual Constraints-
Code runs first and expectations run after, on the DataFrame you return.
Easy to separate *business logic* (in your function) from *data quality checks* (via expectations).